Department of Computer Science and Engineering

Course: CSE447: Cryptography and Cryptanalysis

Semester: Spring 2026

Project Report

Title: SecureApp – A Multi-Algorithm Cryptographic Whistleblower Platform

---

Submitted To: [Instructor Name]

Group No: [Group No]

Section: [Section]

Submission Date: [DD Month 2026]

Group Members

No.	Full Name	Student ID
1	[Member 1 Full Name]	[Member 1 Student ID]
2	[Member 2 Full Name]	[Member 2 Student ID]
3	[Member 3 Full Name]	[Member 3 Student ID]

Table of Contents

1. Introduction and System Overview

This report documents the design, implementation, and security analysis of the CSE447 Lab Project. The system is a secure web application integrating multiple cryptographic protocols as required by the course specification. All encryption algorithms have been implemented from scratch without relying on built-in framework encryption functions.

1.1 Project Overview

SecureApp is a whistleblower and secure communication web platform designed to allow users to anonymously submit tips, publish encrypted investigative reports, exchange end-to-end encrypted messages, and securely share documents. Two user classes are supported: **regular users** (reporters/whistleblowers) and **administrators** (who moderate content, manage keys, and review tips). Core features include:

• Anonymous tip submission with category and urgency tagging

• Encrypted public posts (whistleblower reports) with HMAC integrity verification

• End-to-end encrypted private messaging with optional message expiry

• Dead drops – one-time anonymous document/message delivery channels

• Two-factor authentication via email OTP

• Role-based access control (user / admin)

• Admin dashboard with audit logs, key rotation, failed login tracking, and bulk tip actions

1.2 Technology Stack

Component	Technology
Language	Python 3.12
Web Framework	Flask 3.1.3
Database	SQLite 3 (via Python sqlite3 standard library)
Email (2FA)	Gmail SMTP via smtplib (standard library), TLS on port 587
External Libraries	flask, markupsafe only — zero cryptographic dependencies
Custom Crypto	RSA-1024, ECC ElGamal secp256k1, SHA-256, HMAC-SHA256, Symmetric XOR-CTR — all from scratch
Frontend	Bootstrap 5.3, Orbitron / Rajdhani / JetBrains Mono fonts

1.3 System Architecture Diagram

```
CLIENT BROWSER | | HMAC-signed session cookie v FLASK APP (app.py) auth.py | posts.py | admin.py | messages.py | tips.py | deaddrops.py ... | v key_management.py (KeyManager) RSA encrypt/decrypt | ECC key gen/wrap | HMAC sign/verify | v crypto/ (all from scratch) rsa.py | ecc.py | sha256.py | hmac_custom.py | symmetric.py | utils.py | v database.py -> app.db (SQLite) All fields stored as ciphertext hex strings — zero plaintext
```

2. Login and Registration Module

The system provides secure registration and login flows. New users supply credentials which are validated, encrypted, and persisted. During login, stored encrypted data is retrieved and decrypted for verification.

2.1 Registration Flow

1. User submits username, email, phone (optional), password, and confirm password.

2. Server validates: non-empty required fields, username 3–50 chars, password ≥ 6 chars, passwords match.

3. SHA-256(username.lower()) and SHA-256(email.lower()) computed as lookup hashes — checked for uniqueness without storing plaintext.

4. Username, email, and phone RSA-encrypted using the system public key.

5. A 16-byte random salt is generated; password hashed as SHA-256(salt || password).

6. A fresh ECC (secp256k1) key pair generated for the user; ECC private key RSA-encrypted before storage.

7. HMAC-SHA256 tag computed over (username_hash | email_hash | password_hash) stored as data_hmac.

8. All fields written to the users table — no plaintext stored anywhere.

2.2 Login Flow

1. User submits username and password.

2. SHA-256(username.lower()) used to look up the user row. Unknown usernames are logged to audit_log; generic error shown (prevents enumeration).

3. SHA-256(salt || password) computed and compared to stored password_hash. Mismatches are logged.

4. HMAC integrity of the user record verified before proceeding.

5. 6-digit OTP generated via os.urandom(3), SHA-256 hashed, stored with 5-minute expiry, emailed to RSA-decrypted address.

6. Temporary HMAC-signed token placed in a 2fa_pending session cookie. On OTP submission the server verifies signature, hashes the code, and compares to stored hash.

7. On success: full session token created, HMAC-signed, stored in sessions table with IP binding and 24-hour expiry, set as HttpOnly; SameSite=Strict cookie.

2.3 Implementation Details

Requirement	Implementation Details
Login Module	Username looked up by SHA-256 hash; password verified via SHA-256(salt||password); HMAC integrity
Registration Module	Fields: username (3–50 chars), email, phone (optional), password (≥6 chars), confirm password; uniquen
Data Encrypted Before Storage	username_enc, email_enc, phone_enc → RSA-1024 (system key); ecc_private_key_enc → RSA-1024 w
Data Decrypted on Retrieval	key_manager.decrypt_user_data(ciphertext) calls rsa_decrypt_string() with system RSA private key; ECC

3. User Data Encryption and Decryption

All sensitive user information (e.g., username, email, contact info) is encrypted before storage using asymmetric encryption algorithms implemented from scratch, and decrypted upon retrieval.

3.1 Fields Encrypted

Table	Field	Algorithm
users	username_enc	RSA-1024 (system key)
users	email_enc	RSA-1024 (system key)
users	phone_enc	RSA-1024 (system key)
users	ecc_private_key_enc	RSA-1024 (system key wraps ECC private key)
posts	title_enc, content_enc	ECC ElGamal secp256k1 (user's personal key)
messages	subject_enc, content_enc	ECC ElGamal (recipient's public key)
anonymous_tips	title_enc, content_enc	ECC ElGamal (system ECC key)
documents	file bytes	XOR-CTR (SHA-256 keystream); symmetric key RSA-wrapped
dead_drops	content_enc	ECC ElGamal

3.2 Encryption Algorithm – RSA Implementation (crypto/rsa.py)

The RSA implementation is entirely from scratch with no use of any built-in cryptographic library:

• **Key size:** 1024 bits (two 512-bit primes p and q).

• **Prime generation:** generate_prime(bits) in crypto/utils.py generates a random odd number of the target bit-length and confirms primality with a Miller-Rabin test.

• **Key generation:** n = p × q, $\phi(n) = (p–1)(q–1)$, public exponent e = 65537, private exponent d = e■^{1} mod $\phi(n)$ via the extended Euclidean algorithm.

• **Encryption / Decryption:** c = m^e mod n and m = c^d mod n using Python's pow(base, exp, mod) for efficient modular exponentiation.

• **String encryption:** Data is UTF-8 encoded, prefixed with a 4-byte length header, split into chunks of (key_size – 1) bytes so each chunk < n. Each chunk is encrypted independently and hex-encoded; chunks are joined with ':'.

3.3 Encryption Algorithm – ECC Implementation (crypto/ecc.py)

The ECC implementation uses **ElGamal encryption on the secp256k1 curve**, fully from scratch:

• **Curve:** $y^2 = x^3 + 7 \pmod p$ — secp256k1 (a = 0, b = 7). Standard Bitcoin/Ethereum curve.

• **Curve parameters:** p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F, generator point G at standard secp256k1 coordinates.

• **Point arithmetic:** point_add(), point_double(), and scalar_multiply() (double-and-add) implemented manually.

• **Key generation:** Private key d is a random integer in [1, n). Public key Q = d × G.

• **Koblitz encoding:** Each 30-byte plaintext chunk is encoded as x = m × 100 + j where j (0–99) is the smallest offset for which a valid curve point exists (y^2 is a quadratic residue). y is recovered via modular

square root.

• **ElGamal encryption:** For message point M, random k chosen; ciphertext (C1, C2) = (k×G, M + k×Q).

• **Decryption:** M = C2 – d×C1, since d×C1 = d×k×G = k×Q.

3.4 How Both Algorithms Are Used Differently

Operation	Algorithm	Reason
User profile fields (username, email, phone)	RSA-1024	System-wide encryption; data must be decryptable by the server for admin acces
ECC private key wrapping	RSA-1024	Asymmetric key wrapping — one system RSA key protects all per-user ECC key
Post titles and content	ECC ElGamal (secp256k1)	Per-user encryption; only the post author's key can decrypt — enforces user data
Private messages	ECC ElGamal	Encrypted to recipient's public ECC key — sender cannot decrypt after sending
Anonymous tips, dead drops	ECC ElGamal	Uses system ECC key for admin-accessible decryption without account linkage
Document file bytes	Symmetric XOR-CTR (SHA-256 keystream)	Efficient for large binary payloads; symmetric key RSA-wrapped before storage

4. Password Hashing and Salting

Passwords are never stored in plaintext. A cryptographic hash function combined with a random salt is applied before storage to prevent dictionary and rainbow-table attacks.

4.1 Hashing Algorithm Used

The system uses **SHA-256**, implemented entirely from scratch in **crypto/sha256.py**. SHA-256 was chosen because:

• It produces a 256-bit (32-byte) digest providing strong collision resistance.

• It is the foundational primitive already used throughout the system (HMAC, session token hashing, lookup hashing), keeping the cryptographic surface uniform.

• The custom implementation follows the complete SHA-256 specification: message padding, 64-round compression with schedule words, and the eight initial hash values derived from the square roots of the first 8 primes.

4.2 Salt Generation

Salt is generated using **os.urandom(16)**, producing 16 cryptographically secure random bytes, then hex-encoded to a 32-character string. The salt is stored as plaintext in the **password_salt** column of the users table, separate from password_hash. This is the standard practice — the salt's purpose is uniqueness per user, not secrecy.

4.3 Verification Process

On login, verification proceeds as follows:

1. Retrieve the user row by SHA-256(username.lower()) hash lookup.
2. Read password_salt from the row.
3. Compute candidate_hash = SHA-256(salt_bytes || password_bytes) using the submitted password.
4. Compare candidate_hash with the stored password_hash.
5. If they match, proceed to HMAC integrity verification and then 2FA.

```
# key_management.py def hash_password(self, password, salt): return
sha256_hex(salt.encode('utf-8') + password.encode('utf-8')) # routes/auth.py —
verification password_hash = key_manager.hash_password(password, user['password_salt']) if
password_hash != user['password_hash']: # log failed attempt, reject login
```

5. Two-Factor Authentication (2FA)

The system enforces two-step verification: the user must pass both primary credential validation and a second authentication factor before a session is granted.

5.1 2FA Method

The system uses **Email OTP (One-Time Password)**: a 6-digit code is generated, hashed, and emailed to the user's registered address after successful primary credential verification. The flow is:

1. After correct username/password, a 6-digit code is produced: code = os.urandom(3) % 1,000,000 (zero-padded to 6 digits).

2. SHA-256(code) stored in the two_factor_codes table with a 5-minute expiry (TWO_FA_EXPIRY_MINUTES = 5).

3. The plaintext code is sent via Gmail SMTP (smtplib, TLS on port 587). The email address is obtained by RSA-decrypting the user's email_enc field.

4. A temporary signed token (SHA-256(random) + HMAC signature) is placed in a short-lived 2fa_token HttpOnly cookie, pointing to a 2fa_pending session row binding the user_id.

5. The user submits the code. The server verifies the cookie signature, hashes the submitted code, and compares it against the stored hash.

6. On success: the temporary session is deleted and a full permanent session cookie is issued.

5.2 Code Snippet

```
# routes/auth.py — 2FA code generation code = str(int.from_bytes(secure_random_bytes(3), 'big') % 1000000).zfill(6) code_hash = sha256_hex(code.encode('utf-8')) expires_at = (datetime.utcnow() + timedelta(minutes=TWO_FA_EXPIRY_MINUTES)).isoformat() db.create_2fa_code(user['id'], code_hash, expires_at) # Decrypt email and send OTP user_email = key_manager.decrypt_user_data(user['email_enc']) send_2fa_code(user_email, code) # Temporary signed token for the 2FA step temp_token = sha256_hex(secure_random_bytes(16)) temp_sig = key_manager.sign_session_token(temp_token) response.set_cookie('2fa_token', f'{temp_token}.{temp_sig}', httponly=True, samesite='Strict', max_age=TWO_FA_EXPIRY_MINUTES * 60) # routes/auth.py — verification code_hash = sha256_hex(code.encode('utf-8')) if not db.verify_2fa_code(user_id, code_hash): flash('Invalid or expired verification code.', 'danger')
```

6. Key Management Module

A dedicated Key Management Module (key_management.py) handles the full lifecycle of cryptographic keys: secure generation, storage, and rotation.

6.1 Key Storage Security

The **KeyManager** class manages four keys:

Key	Storage	Protection
RSA public key	keys/system_rsa.json	Public key — no secrecy needed
RSA private key	keys/system_rsa.json	JSON file on server filesystem; all user data unreadable without it
HMAC session key (32 B)	keys/hmac_keys.json	Used to sign/verify session tokens
HMAC data key (32 B)	keys/hmac_keys.json	Used for data integrity MACs on all records

User ECC private keys are **never stored in plaintext**: they are RSA-encrypted with the system public key (rsa_encrypt_string(priv_str, self.rsa_public_key)) and stored as ecc_private_key_enc in the database. Decryption requires the system RSA private key, which exists only server-side.

6.2 Key Rotation Policy

The system provides **admin-triggered key rotation** accessible via /admin/keys. When rotation is triggered:

1. A new RSA-1024 key pair is generated with generate_rsa_keypair(RSA_KEY_BITS).

2. All users' ecc_private_key_enc fields are re-encrypted: old RSA private key decrypts them; new RSA public key re-encrypts them.

3. All user profile fields (username_enc, email_enc, phone_enc) similarly re-encrypted: decrypt with old key → encrypt with new key → update in database.

4. New keys replace the old ones in keys/system_rsa.json.

5. A key_rotation_log entry is recorded with timestamp and the admin user ID who triggered it.

6. HMAC keys can also be independently rotated. After rotation, new keys are used for all future HMAC computations.

This guarantees that existing encrypted records are never broken by a key rotation event.

7. Post and Profile Management

Users can create, view, and edit posts, as well as view and update their profiles. All post and profile data is automatically encrypted before storage and decrypted on retrieval.

7.1 Post Module

Create: User submits a title (max 200 chars) and content (max 5000 chars). The system uses the user's ECC public key to encrypt both fields with ecc_encrypt_string(). A HMAC-SHA256 tag over (title_enc, content_enc, user_id) is computed and stored as data_hmac. Optionally an uploaded document is encrypted with a symmetric XOR-CTR key, itself RSA-wrapped before storage.

Read: The user's ECC private key (decrypted via RSA from ecc_private_key_enc) is used with ecc_decrypt_string(). Before decryption, HMAC is verified; if it fails the post is flagged [Integrity Check Failed] instead of revealing content.

Update: The /posts/<id>/edit route re-encrypts the new title and content with the same ECC public key and recomputes the HMAC.

Field	Encrypted With
title_enc	ECC ElGamal (user's public key)
content_enc	ECC ElGamal (user's public key)
data_hmac	HMAC-SHA256 over title_enc + content_enc + user_id

7.2 Profile Module

View: The profile.py route decrypts username_enc, email_enc, and phone_enc via key_manager.decrypt_user_data() (RSA). It also computes a **key fingerprint**: the first 16 hex characters of SHA-256(ecc_public_key) formatted as XXXX:XXXX:XXXX:XXXX.

Update: The /profile/edit route accepts new username, email, phone, and optionally a new password. Updated fields are RSA-encrypted before storage and data_hmac is recomputed.

Field	Encrypted With
username_enc	RSA-1024 (system key)
email_enc	RSA-1024 (system key)
phone_enc	RSA-1024 (system key)

7.3 Screenshots

[Insert screenshots of the post creation page at /posts/create, the post listing page at /posts, and the profile management page at /profile/edit.]

8. Data Storage Security

All critical data — user information, posts, and cryptographic keys — is stored in encrypted form to prevent plaintext access even in the event of a database compromise.

8.1 Evidence of Encrypted Storage

The SQLite database app.db stores all sensitive fields as hex-encoded ciphertext strings. A direct inspection of the users table in any SQLite browser reveals:

• **username_enc:** Long colon-separated hex string (RSA-chunked ciphertext)

• **email_enc:** Long colon-separated hex string (RSA-chunked ciphertext)

• **password_hash:** 64-character SHA-256 hex digest (separate password_salt column)

• **ecc_public_key:** Serialized point coordinates (two hex integers separated by ':')

• **ecc_private_key_enc:** RSA-encrypted ECC private key (colon-separated hex chunks)

• **data_hmac:** 64-character HMAC-SHA256 hex digest

The posts table stores title_enc and content_enc as long hex strings (ECC ElGamal ciphertext tuples: c1x,c1y,c2x,c2y per chunk, separated by ';'). No column in any table stores plaintext user-supplied content.

[Insert screenshot of raw database records from DB Browser for SQLite or equivalent tool showing ciphertext, not plaintext.]

9. Message Authentication Code (MAC)

Message Authentication Codes (MACs) are used to verify the integrity of stored data and detect any unauthorized modifications.

9.1 MAC Algorithm Used

The system uses **HMAC-SHA256**, implemented from scratch in **crypto/hmac_custom.py**. HMAC was chosen over CBC-MAC because:

• HMAC is provably secure as long as the underlying hash is secure, without requiring a block cipher.

• It handles variable-length inputs (post content, user data) naturally, which CBC-MAC handles less cleanly.

• The custom SHA-256 implementation was already available as the underlying primitive.

The standard HMAC construction is followed exactly:

```
HMAC(K, m) = SHA-256( (K' XOR opad) || SHA-256( (K' XOR ipad) || m ) ) where K' = key padded/hashed to 64 bytes (SHA-256 block size) ipad = 0x36 repeated 64 times opad = 0x5C repeated 64 times
```

Verification uses constant-time byte-by-byte XOR comparison to prevent timing attacks. Two independent 32-byte HMAC keys are maintained by KeyManager: **hmac_session_key** (signs session tokens) and **hmac_data_key** (data integrity tags).

9.2 Integrity Verification Flow

HMAC verification is performed:

1. On every user post read (/posts, /posts/): stored data_hmac verified against (title_enc, content_enc, user_id) before decryption. Failure displays [Integrity Check Failed].

2. On every login: user record data_hmac verified against (username_hash, email_hash, password_hash). Failure aborts login.

3. On every HTTP request: session cookie token.signature verified by recomputing HMAC(hmac_session_key, token) with constant_time_compare().

4. On anonymous tips and messages: data_hmac verified during read operations before content is displayed.

10. Role-Based Access Control (RBAC)

Role-Based Access Control defines distinct privilege levels for Administrators and Regular Users, ensuring that sensitive operations are restricted appropriately.

10.1 Roles Defined

Two roles are stored in the **users.role** column, enforced by a SQL CHECK constraint (role IN ('user', 'admin')):

• **User:** Regular authenticated member. Can create and manage own posts, submit anonymous tips, exchange encrypted messages, use dead drops, and manage their profile.

• **Admin:** Privileged operator. Has all user capabilities plus full administrative control — user management, key rotation, tip review, post moderation, and audit log access. Enforced by the @admin_required decorator in routes/__init__.py (aborts HTTP 403 if role != 'admin').

10.2 Permission Matrix

Operation / Resource	Admin	Regular User
View own profile	✔	✔
Edit own profile	✔	✔
Create / Edit own posts	✔	✔
Submit anonymous tips	✔	✔
Send / receive encrypted messages	✔	✔
Create / access dead drops	✔	✔
View all user accounts	✔	✘
Change user roles	✔	✘
Delete any post	✔	✘
View / manage all tips	✔	✘
Bulk update tip status	✔	✘
Manage / rotate crypto keys	✔	✘
View audit logs / failed logins	✔	✘
Change post publication status	✔	✘

11. Secure Session Management

Authentication tokens and session identifiers are managed securely to prevent session hijacking, fixation, and replay attacks.

11.1 Token Signing / Verification

Sessions are managed entirely with custom cryptography — no Flask session, JWT, or third-party auth libraries are used.

Token creation:

1. 256-bit random token: token = SHA-256(os.urandom(32)).
2. CSRF token: csrf_token = SHA-256(os.urandom(16)).
3. Token HMAC-signed: signature = HMAC(hmac_session_key, token).
4. Cookie value sent to browser: {token}.{signature}.
5. Database stores SHA-256(token) as token_hash (never the raw token), plus user_id, ip_address, SHA-256(User-Agent), csrf_token, and expires_at (24 hours).

Verification on every request (middleware in app.py):

1. Split cookie into token and sig.
2. Recompute expected_sig = HMAC(hmac_session_key, token).
3. Compare using constant_time_compare() — byte-by-byte XOR to prevent timing attacks.
4. Hash token → look up in sessions table.
5. Check expiry against datetime.utcnow().
6. Verify ip_address matches request.remote_addr (IP binding to prevent cookie theft).
7. Load user into flask.g for the duration of the request.

Additional protections:

• Cookies set HttpOnly=True, SameSite=Strict to prevent XSS and CSRF access.

• CSRF tokens injected into all state-changing forms and verified server-side.

• Expired sessions pruned by db.cleanup_expired_sessions() triggered probabilistically on ~1% of requests.

• Users can view and terminate all active sessions from the /sessions page.

12. GitHub Repository and Project Structure

Field	Details
GitHub Repository URL	https://github.com/monirakib/SecureApp

12.1 Repository Structure

```
SecureApp/ |-- app.py # Entry point, session middleware, blueprint registration |-- config.py # Configuration (paths, key sizes, SMTP, upload limits) |-- database.py # All SQLite CRUD operations |-- key_management.py # KeyManager: RSA/ECC/HMAC key lifecycle |-- email_sender.py # Gmail SMTP 2FA email delivery |-- codenames.py # Anonymous codename generator for tips |-- requirements.txt # flask, markupsafe only (no crypto dependencies) | |-- crypto/ # All cryptographic primitives (from scratch) | |-- rsa.py # RSA-1024: key gen, encrypt/decrypt strings | |-- ecc.py # ECC ElGamal secp256k1: key gen, encrypt/decrypt | |-- sha256.py # SHA-256 hash function | |-- hmac_custom.py # HMAC-SHA256 | |-- symmetric.py # XOR-CTR symmetric encryption (SHA-256 keystream) | +-- utils.py # Prime gen, mod_inverse, secure random | |-- routes/ # Flask blueprints | |-- auth.py # Register, login, 2FA, logout | |-- posts.py # Create/read/edit posts (ECC encrypted) | |-- profile.py # View/edit profile, ECC key fingerprint | |-- admin.py # Admin dashboard, user/post/tip mgmt, key rotation | |-- feed.py # Public post feed with category/urgency filters | |-- messages.py # Encrypted private messaging with expiry | |-- tips.py # Anonymous tip submission and admin review | |-- deaddrops.py # One-time encrypted dead drops | |-- friends.py # Friend/contact management | +-- sessions.py # Active session management | |-- templates/ # Jinja2 HTML templates (30+ pages) |-- static/ # style.css, animations.js |-- keys/ # system_rsa.json, hmac_keys.json (server-side only) +-- uploads/ # Encrypted document files (.enc)
```

12.2 README Overview

The repository README covers:

• **Project description:** Secure whistleblower platform for CSE447 with all crypto implemented from scratch.

• **Requirements:** Python 3.12+, pip install flask markupsafe.

• **Setup:** Clone repo → pip install -r requirements.txt → set EMAIL_APP_PASSWORD environment variable for 2FA emails → py app.py.

• **First run:** RSA-1024 key pair and HMAC keys are auto-generated on first startup and saved to keys/.

• **Database:** SQLite database app.db is auto-initialized on first run via init_db().

• **Admin setup:** First user can be promoted to admin via the admin role-change UI or directly in app.db.

13. Conclusion

The SecureApp project successfully demonstrates a fully functional secure web application built with cryptographic algorithms implemented entirely from scratch. The primary challenges were correctly implementing the mathematical foundations of RSA (modular exponentiation, Miller-Rabin primality testing) and ECC (point arithmetic on secp256k1, Koblitz message encoding), as small implementation errors can produce silently incorrect results or break decryption across sessions. Another significant challenge was designing the session management and 2FA flow without any third-party auth libraries, requiring careful attention to token signing, constant-time comparison, IP binding, and code expiry race conditions.

Through this project, the group gained deep practical insight into how cryptographic primitives compose into a real security architecture — understanding not just the algorithms in isolation, but how key wrapping, HMAC integrity chains, and RBAC work together to protect data at rest and in transit. The final system implements seven cryptographic modules across 30+ routes and templates, with no plaintext sensitive data stored anywhere in the database.